

Runtime Analysis of Cartesian Genetic Programming in Evolving Boolean Functions

Duc-Cuong Dang¹, Roman Kalkreuth², and Andre Opris¹

¹University of Passau, Passau, Germany

²RWTH Aachen University, Aachen, Germany

Abstract

Cartesian Genetic Programming (CGP) is among the practical and popular forms of Genetic Programming as it uses a graph-based representation of programs. This paper presents a first runtime analysis of CGP in evolving Boolean functions using complete training sets. We prove an asymptotic bound $O(nD^5)$ for the expected number of fitness evaluations of CGP to construct a conjunction of n inputs using at most $D \geq n-1$ binary gates, a minimal function set, and even with a strict survival selection. When the non-strict selection is used, the bound is improved to $O(nD^4)$. Our analysis reveals interesting characteristics of CGP induced search, which have been only observed empirically. In particular, enabling the acceptance of equally good solutions, including those with connected gates non-contributing to fitness, can lead to a speedup, and consequently a better asymptotic time bound. In contrast to conjunctions, we also prove a negative result which shows that CGP requires exponential time to evolve an exclusive disjunction. Experiments evolving conjunctions complement our theoretical findings. The use of incomplete training sets is found to further reduce the average number of fitness evaluations while maintaining a good level of generalisation.

1 Introduction

Genetic Programming (GP) is a search paradigm originally proposed for the automated discovery of computer programs. GP leverages evolutionary search using selection and variation to evolve programs given a specification, constraints, and input-output examples. Early work by Cramer [4] and Dickmanns et al. [5] applied Genetic Algorithms (GA) to evolve computer programs. However, Koza [17, 18] later introduced a tree-based form of GP to evolve symbolic expressions, commonly implemented as LISP S-expressions at that time, which widely popularised the concept of GP. The traditional form of GP therefore uses tree data structures to model programs, and this is commonly referred to as Tree-based GP (TGP). The leaf nodes represent the inputs of the program and are assigned elements from a *terminal set* T , which consists of variables or constants. The root and internal nodes compute functions from a *function set* F , taking the arguments from the outputs of their respective child nodes. After evaluation, the value produced at the root node constitutes the final output of the modeled program. Over time, GP has been extended to incorporate alternative program representations, including linear sequences [32, 31], graph-based structures [33, 26], and rule-based models [34].

In this work, we are interested in Cartesian Genetic Programming (CGP) [25], which is, unlike TGP, a graph-based form of GP. CGP is a well established methodology in logic synthesis and evolvable hardware [40], with many successful applications such as in designing high order combinational circuits [39], complex approximate circuits [41], and cryptographic functions [11]. These successes are believed to be due to the high versatility of the graph-based representation. Learning Boolean functions was also among the motivating applications for the introduction of CGP [26]. However, it remains poorly understood when and why even simple CGP algorithms perform well in this problem domain, and a solid theoretical foundation for CGP in evolving Boolean functions is still missing.

Our contribution: Inspired by the work of Mambrini and Oliveto [22] on TGP, we initiate a rigorous analysis of CGP for evolving Boolean functions by analysing the runtime of the (1+1) CGP and the strict variant (1+1) CGP*. We consider the popular single active-gene mutation (SAM) [9] and the minimal function sets. For constructing conjunctions of n variables with $D \geq n - 1$ AND gates and the complete training set, we obtain expected runtimes of $\mathcal{O}(nD^4)$ and $\mathcal{O}(nD^5)$, respectively, using fitness-level arguments and Markov chain analysis. The faster performance of (1+1) CGP over (1+1) CGP* stems from its non strict selection, which accepts equally fit solutions with redundant active gates, a beneficial effect previously observed only empirically [35]. Prior counterpart TGP results [19, 22] make no distinction between strict and non strict selection, and in comparison, CGP is slower but guarantees to only use $\mathcal{O}(D \log D)$ space. The original CGP [26] was not designed for speeding up TGP, but to offer a more general representation of programs with natural support for Boolean circuit applications. A CGP program in the Boolean domain is a logic circuit. The aim of our paper is not about showing a superior performance, but rather to understand how and when polynomial performance guarantees can be achieved.

In contrast to conjunctions, we prove that (1+1) CGP requires exponential time with exponentially high probability, to construct an exclusive disjunction of n inputs out of $(1 + d)n \geq D \geq n - 1$ XOR gates for some $d < 1$ and using the complete training set. Here we apply the negative drift theorem [29]. Experiments on conjunctions support the theory, showing clear gains by using non strict selection. We also conduct experiments using incomplete static training sets, and observe that this setting further reduces the number of fitness evaluations required by CGP while maintaining a good level of generalisation.

Related work: From the empirical side, several studies have examined search behaviour of CGP, primarily focusing on characteristics such as neutral search [43], redundancy [24], bloat [35] and evolvability in the fitness landscapes induced by logic synthesis problems [43]. Moreover, based on the results of empirical studies, it has been demonstrated and hypothesised that CGP-driven program search benefits from high levels of redundancy, enabled by the use of very large genotypes [24, 35]. This also includes handling neutral fitness landscapes by neutral genetic drift (NGD), which has been shown to help escape local optima [43, 36]. A step forward in understanding the complexity characteristics of CGP induced search in logic synthesis was made by the proposal of the General Boolean Function Suite (GBFS) [14]. This is a diverse benchmark of different types of Boolean functions commonly used to evaluate GP methods.

Mambrini and Oliveto [22] were the first to rigorously analyse GP in evolving Boolean functions by considering the same TGP systems as [27, 16] using the minimal function sets. On the complete training set, they showed that the (1+1) TGP and (1+1) TGP* with the single-step (local) mutation constructs a conjunction of n inputs in $\mathcal{O}(n \log n)$ expected fitness evaluations. On a static incomplete training sampled uniformly at random of $s \in \text{poly}(n)$ rows, the time bound to fit the training set is reduced to $\mathcal{O}(\log n)$. Using the negative drift theorem, they also showed that the (1+1) TGP requires exponential time to construct an exclusive disjunction of n variables. Other settings were also considered in their paper, such as constructing exclusive disjunction with incomplete training set, and conjunction with negated input variables. These results were later extended for the algorithms with the multi-step (global) mutation in Lissovoi and Oliveto [19, 20], which added the generalisation ability guarantee for conjunctions on incomplete training sets, along with many other results. Doerr et al. [7, 8] pushed this line of research further by considering the settings of non-minimal function sets, unknown target function size, and introduced a new tool, called the super-multiplicative theorem, for their analysis.

For CGP, we are unaware of any theoretical research in learning Boolean functions. The only work on runtime analysis of CGP that we know of is from Kalkreuth and Droschinsky [12] on one function node. Finally, the book chapter [21] summarises the current state-of-the-art theoretical research on GP.

2 Preliminaries

We use the asymptotic notation with symbols Ω , $\mathcal{O}$, Θ [1]. The sets of integer and natural numbers are $\mathbb{Z}$ and $\mathbb{N}$ respectively. For $n \in \mathbb{N}$, define $[n] := \{1, \dots, n\}$ and $[n]_0 := [n] \cup \{0\}$. The natural logarithm is denoted

$\ln(\cdot)$ and that of base-2 is $\log(\cdot)$. We use H_n to denote the n -th harmonic number, i.e. $\ln(n+1) < H_n := \sum_{i=1}^n \frac{1}{i} < 1 + \ln(n)$, thus $H_n = \mathcal{O}(\log n)$. Let $\text{Li}_s(z) := \sum_{i=1}^{\infty} \frac{z^i}{i^s}$ be the polylogarithm (or Jonquière's) function for $s, z \in \mathbb{C}$, i.e. extended by analytic continuation. Some special cases are $\text{Li}_{-2}(z) = \sum_{i=1}^{\infty} i^2 z^i = \frac{z(1+z)}{(1-z)^3}$ for $|z| < 1$, and $\text{Li}_s(1) = \sum_{i=1}^{\infty} \frac{1}{i^s} =: \zeta(s)$ which is known as the Riemann's Zeta function. Note that $\zeta(2) = \pi^2/6$ and $\zeta(4) = \pi^4/90$. The logic AND, OR and exclusive OR are denoted $\wedge$, $\vee$ and $\oplus$ respectively. The uniform distribution over a set A is denoted $\text{Unif}(A)$ and the geometric distribution with parameter $p \in [0, 1]$ is denoted $\text{Geom}(p)$. For two random variables X, Y , $X \succeq Y$ means that X stochastically dominates Y , that is, for any $a \in \mathbb{R}$ it holds $\Pr(X \leq a) \leq \Pr(Y \leq a)$.

2.1 Cartesian Genetic Programming

CGP is capable of modelling programs with N_{in} inputs and N_{out} outputs, and uses N_{func} (internal) function nodes. The functions that can be used to compute the output of those nodes are defined in a lookup table of K entries indexed from 0 to $K-1$. The function specified in each entry has an arity of at most k . A (valid) program for such a CGP is then represented by a directed acyclic graph $G = (V, E)$, referred to as the *phenotype*, which defines how the outputs are computed by composing functions of the lookup table and using the respective inputs. The inputs and the lookup table model the same concepts as the terminal set T and the function set F in Tree-based GP, thus any TGP program always has an equivalent CGP program.

The original form of CGP [26] was proposed to arrange the function nodes on a grid-like structure similar to an FPGA circuit. However, we consider the modern (simplified) approach in which the nodes are arranged in a single line, thus V can be indexed from 0 to $N-1$ where $N := N_{\text{in}} + N_{\text{func}} + N_{\text{out}}$ or simply $V = [N-1]_0$, i.e. starting with the input nodes N_{in} , then the function nodes N_{func} , and finally the output ones N_{out} . Each input node has no incoming edges (or connections) while each output node has exactly one coming from a non-output node, i.e. with index in $[N - N_{\text{out}} - 1]_0$. Each function node is labelled with a function of the lookup table and has exactly k incoming edges from nodes of strictly smaller indices, thus from the inputs or from the outputs of the preceding function nodes. These edges represent the sequence of arguments that the function at the node takes, thus there is an ordering among them. If the arity of the corresponding function in the lookup table is smaller k , then only some first edges matter.

It is possible for a CGP program to ignore some of the function nodes, which are referred to as *inactive* (function) nodes. More generally, a non-terminal node of V is called *active* if there is a path from this specific node to one of the output nodes, otherwise it is called *inactive*. By this definition, output nodes are always active. The *genotype* (or the encoding) of G is a vector of $N_{\text{func}}(k+1) + N_{\text{out}}$ integers. The sequence is divided into $N_{\text{func}} + N_{\text{out}}$ blocks indexed from N_{in} to $N-1$, and these indices match those nodes in the phenotype that the blocks encode. A block with index $i < N_{\text{in}} + N_{\text{func}}$ (a so called function block) contains $k+1$ integers (or genes), the first, referred to as the *function gene*, is a number in $[K-1]_0$ (pointing to an index in the lookup table) while the rest, referred to as *function connection genes*, are integers in $[i-1]_0$. Blocks with a larger index than i (output blocks) only contain one integer in $[N - N_{\text{out}} - 1]_0$, called the *output connection gene*. Thus the set of all genotypes (all valid CGP programs), denoted by $\mathcal{X}$, is an integer-valued space. Unlike TGP, CGP therefore has a fixed space complexity to store a program as stated in the lemma below. Figure 1 gives an example of a CGP program with its phenotype representation and the corresponding genotype encoding.

Lemma 1. *Only $(N_{\text{func}}(k+1) + N_{\text{out}}) \log(1 + \max\{K, N_{\text{in}} + N_{\text{func}}\})$ storage space is required to store a CGP program.*

Proof. The program is encoded by a vector of $N_{\text{func}}(k+1) + N_{\text{out}}$ integers. Each integer is bounded from above by $\max\{K, N_{\text{in}} + N_{\text{func}}\}$, thus at most $\log(1 + \max\{K, N_{\text{in}} + N_{\text{func}}\})$ bits are required to encode it. The total number of bits is $(N_{\text{func}}(k+1) + N_{\text{out}}) \log(1 + \max\{K, N_{\text{in}} + N_{\text{func}}\})$. $\square$

We consider the simplest CGP algorithm the so-called (1+1) CGP (Algorithm 1 for minimisation). Starting from a program initialised uniformly at random, in each iteration a new offspring program y is generated by mutating the current one. The offspring will replace the parent only if it has better or equally good fitness. Here

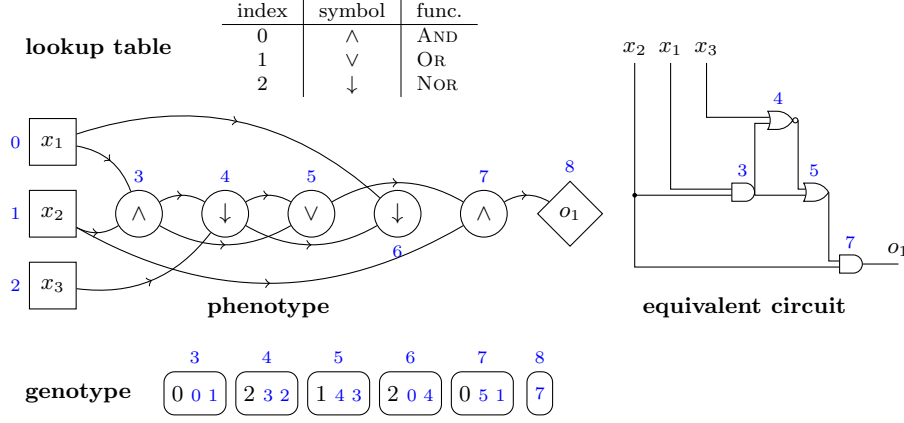


Figure 1: Example of a CGP program that models a Boolean function. Node 6 is a so-called inactive node since no path touching this node leads to the output.

$f(x)$ is referred to as the fitness of a program x which is problem-dependent and assumed given. By convention in runtime analysis, we let the algorithm run infinitely then estimate the expected time to reach some target state. Like TGP [22], we also refer to the variant of the (1+1) CGP where strict comparison is used instead in Line 4 as the (1+1) CGP*.

Mutations in CGP can alter any gene (component) of the genotype (vector) to a new value uniformly at random while respecting the constraint on their range, this is done with the help of the procedure `NEWVALUE` which is summarised in Algorithm 2. When enforcing the change of the gene value is not possible, for example if only one function is specified in the lookup table then the function genes are fixed to zero and cannot be mutated, the old value is returned instead.

The original paper [23] proposed *point mutation* operators in which a fixed or random number of genes (mutation points) are chosen uniformly at random for alternation. Nowadays, the so-called *single active-gene mutation* (SAM) [9] is more popular and is our focus. SAM is parameter-free and is described in Algorithm 3. It repeatedly modifies genes chosen uniformly at random until an active node is modified. The detection of active nodes can be done during the program evaluation, so we assume that there is a function `ISACTIVE` that can tell whether a given gene belongs to an active node or not in an evaluated program.

2.2 Learning Boolean Functions with GP

Learning Boolean functions or, more generally, logic circuits with heuristic methods is part of the wider field of logic synthesis [10]. In this section, we describe how the learning is evaluated for any GP.

The problem of learning a Boolean function $g: \text{BOOL}^n \rightarrow \text{BOOL}$ on n inputs where $\text{BOOL} := \{\text{TRUE}, \text{FALSE}\}$ is to compose a program/circuit a set of elementary logic gates to exactly match or to approximates g using a set S of input-output examples of g referred to as the *training set*. To accommodate the construction of S by random sampling, we rather consider S as a multi-set, each of its elements is called a *row* which consists of a pair of example input assignment $\alpha \in \text{BOOL}^n$ and the desired output $g(\alpha) \in \text{BOOL}$. Formally, the set of input variables is $T := \{x_i \mid i \in [n]\}$ and coincides with the GP terminal set. Then each $\alpha: T \rightarrow \text{BOOL}$ is a mapping of every input to a Boolean value. Like g , a given candidate program y is also a function $x_1, \dots, x_n \mapsto y(x_1, \dots, x_n)$

Algorithm 1: The (1+1)-CGP on the space $\mathcal{X}$ of programs.

```

1  $x \sim \text{Unif}(\mathcal{X})$ 
2 for iter := 1  $\rightarrow \infty$  do
3    $y := \text{mut}(x)$ 
4   if  $f(y) \leq f(x)$  then  $x := y$ 
```

Algorithm 2: NEWVALUE(x, i).

```
1  $b := \lfloor i/(k+1) \rfloor$  // block index
2  $j := i \bmod (k+1)$  // index inside a block
3 if  $i < N_{\text{func}}(k+1)$  then
4   if  $j = 0$  then  $S := [K-1]_0$  // valid functions
5   else  $S := [N_{\text{in}} + b - 1]_0$  // valid input connections
6 else
7    $S := [N_{\text{in}} + N_{\text{func}} - 1]_0$  // valid output connections
8 if  $|S \setminus \{x[i]\}| \geq 1$  then
9    $S := S \setminus \{x[i]\}$  // avoid the old value if possible
10  $v \sim \text{Unif}(S)$ 
11 return  $v$ 
```

Algorithm 3: MUTATIONSAM(x).

```
1 Copy  $x$  to  $y$ 
2  $\text{active} := \text{false}$ 
3 while  $\neg \text{active}$  do
4    $i \sim \text{Unif}([N_{\text{func}}(k+1) + N_{\text{out}} - 1]_0)$ 
5    $y[i] := \text{NEWVALUE}(y, i)$ 
6    $\text{active} := \text{ISACTIVE}(x, i) \wedge (y[i] \neq x[i])$ 
7 return  $y$ 
```

on the inputs. To calculate the fitness of y , the number of mismatched outputs of y on S is counted, that is,

$$f(y) := |\{(\alpha, g(\alpha)) \in S \mid y(\alpha(x_1), \dots, \alpha(x_n)) \neq g(\alpha(x_1), \dots, \alpha(x_n))\}|.$$

Thus the smaller $f(y)$ is the closer y is to fully fit the training set S .

In the so-called synthesis *with the complete training set*, the complete specification of g is available and is used to evaluate candidate programs, that is, $S = \mathcal{K}_g$ where $\mathcal{K}_g := \{(\alpha, g(\alpha(x_1), \dots, \alpha(x_n)))\}_{\alpha \in \text{BOOL}^n}$ is the complete truth table of g . Since the truth table grows exponentially in n , i.e. $|\mathcal{K}_g| = 2^n$, in practice to reduce the computational time of each fitness evaluation for large n , only a fraction of $\mathcal{K}_g$ is sampled and used as training set S . This is referred to as synthesis *with an incomplete training set*. We assume that S is sampled once and remains unchanged during evolution, i.e. S is called a *static* training set.

We only consider the complete training set in our theoretical analysis, the reason is discussed in the last section of the paper. However, in experiments, incomplete training sets are considered. Similarly to [7, 22], we define the *generalisation error* $\varepsilon_g(y)$ of a program y in modelling a target function g as the probability that y returns a mismatched output of a row selected uniformly at random from $\mathcal{K}_g$, i.e. $\varepsilon_g(y) := \Pr(y(\alpha(x_1), \dots, \alpha(x_n)) \neq g(\alpha(x_1), \dots, \alpha(x_n)))$ where $\alpha \sim \text{Unif}(\text{BOOL}^n)$. The *generalisation ability* of y is the complement of its generalisation error, i.e. $G_g(y) := 1 - \varepsilon_g(y)$. Thus y fully meets the specification of g if and only if $G_g(y) = 1$. A value of $G_g(y)$ close to 1 means that y generalises g well while a value close to $1/2$ means that it is only as good as a random guess. A program y with $G_g(y)$ close to 0 can be turned to $\bar{y}$ with $G_g(\bar{y})$ close to 1, thus y and $\bar{y}$ are equally informative of g .

The exact computation of $G_g(y)$ is of course expensive, however it can be approximated by considering a *validation set* (also called *test set*) S' which is sufficiently large and is created independently from S by sampling the rows of $\mathcal{K}_g$ uniformly at random with replacement. The fitness of y evaluated on S' , for which y was not evolved to fit, gives an estimation of $\varepsilon_g(y)$ on S' , and is denoted $\hat{\varepsilon}(y, S')$. The quantity $\hat{G}(y, S') := 1 - \hat{\varepsilon}(y, S')/|S'|$ is the maximum likelihood estimator, and hence an approximation, of $G_g(y)$.

3 Analysis of Evolving Conjunctions

The first target Boolean function that we analyse is the AND_n function of $n \geq 2$ inputs, i.e. $\text{AND}_n(x_1, \dots, x_n) = x_1 \wedge x_2 \wedge \dots \wedge x_n$, that must be built from at least $n - 1$ binary AND gates. In GP terminology, we have the

function set $F = \{\text{AND}\}$ and the terminal set $T := \{x_i \mid i \in [n]\}$. Similar to the naming convention for the MAX problem [16], but here without restricting the tree depth as it is not part of the problem definition, we adapt the name $\text{AND}_n\text{-}\{\text{AND}\}\text{-}\{x_i \mid i \in [n]\}$ for this logic synthesis problem. Regarding the complete training set of this function, only one row of $\mathcal{K}_{\text{AND}_n}$ has $\text{AND}_n(\alpha(x_1), \dots, \alpha(x_n)) = \text{TRUE}$ when $\alpha(x_1) = \dots = \alpha(x_n) = \text{TRUE}$, all the other rows have the desired output FALSE.

To implement this synthesis in CGP, it suffices to set $N_{\text{in}} = n$, $N_{\text{out}} = 1$, use a lookup table with the single binary function AND (i.e., $K = 1$ and $k = 2$), and choose $N_{\text{func}} \in \mathbb{N}$ such that $N_{\text{func}} \geq n - 1$. It follows from [22] that the fitness of programs for $\text{AND}_n\text{-}\{\text{AND}\}\text{-}\{x_i \mid i \in [n]\}$ is monotonically improved (here decreased for the minimisation problem) as a function of the number of input variables used/included (connected to active nodes in case of CGP).

Lemma 2 (Lemma 1 in [22]). *On $\text{AND}_n\text{-}\{\text{AND}\}\text{-}\{x_i \mid i \in [n]\}$, the truth table of a program currently using i distinct input variables differs from the target truth table of $x_1 \wedge x_2 \wedge \dots \wedge x_n$ at exactly $f_i = 2^{n-i} - 1$ rows.*

We show that (1+1) CGP and (1+1) CGP* with $N_{\text{func}} := D \geq n - 1$, using SAM mutation and the complete training set, synthesise the target function of $\text{AND}_n\text{-}\{\text{AND}\}\text{-}\{x_i \mid i \in [n]\}$ in expected time polynomial in D . In particular, for $D \in [n - 1, \infty) \cap O(n)$, the expected time is $\mathcal{O}(n^6)$ and the space required is $\mathcal{O}(n \log n)$.

Theorem 3. *The (1+1) CGP and (1+1) CGP* fit the complete training set of $\text{AND}_n\text{-}\{\text{AND}\}\text{-}\{x_i \mid i \in [n]\}$ using $N_{\text{func}} := D \geq n - 1$, and SAM mutation in $\mathcal{O}(nD^5)$ fitness evaluations in expectation and uses $\mathcal{O}(D \log D)$ space to store its programs. If $D \geq 10$, the probability that the number of fitness evaluations exceeds $(1 + \delta)20\pi^2(n - 1)D^5$ for any $\delta \geq 1$ is at most $e^{-\delta/3}$.*

Proof. We apply the fitness level method to bound the expected running time. The search space $\mathcal{X}$ of valid programs is partitioned into n fitness levels and the (1+1) CGP is said to be at level i if its current program has i inputs connected to the output via its active function nodes. It follows from Lemma 2, and because $2^{n-i} - 1$ is a monotone function, that the algorithm cannot fall down to a level below its current level. We then estimate for each level $i \in [n - 1]$ a lower bound s_i on the probability of leaving the current level to a higher one.

Our arguments will only consider strict improvement steps thus the implied result holds for both (1+1) CGP and (1+1) CGP*. Another way to have $D \geq n - 1$ is to assume that $D = \alpha(n - 1)$ for some $\alpha \geq 1$.

For a function node to be active, either an input edge of another function node (after it) or the output connection must be connected to it. So, for a program with i active function nodes at least $i - 1$ input edges of these nodes have been already used to maintain the node activeness, here the -1 takes into account the output connection. This leaves at most $2i - (i - 1) = i + 1$ of the remaining input edges of these nodes to connect to the inputs, in other words the program can be at most at level $i + 1$. Conversely, a program at level i must have at least $i - 1$ active function nodes. We consider the following cases.

Case 1: If there is at least one input edge of an active function node that can be redirected to an unused input, we then estimate the probability of this redirection in one iteration of the while-loop in SAM. The component associated with that input edge is selected for modification with probability $\frac{1}{3D+1}$, then the modification points the edge to one of the $n - i$ unused inputs with probability $\frac{n-i}{D+n-1} \geq \frac{n-i}{2D}$. Thus, the probability of leaving the current level in this case is at least $\frac{n-i}{(3D+1)(2D)} = \frac{(n-i)(D-i+1)}{(3D+1)(2D)(D-i+1)} \geq \frac{\alpha(n-i)^2}{(3D+1)(2D^2)}$. The last inequality holds because $D - i + 1 \leq D$ and $D - i + 1 = \alpha(n - 1) - i + 1 = \alpha n - (\alpha i - \alpha i + \alpha + i - 1) = \alpha n - (\alpha i - (\alpha - 1)(i - 1)) \geq \alpha n - \alpha i$ for $i \geq 1$ and $\alpha \geq 1$.

Case 2: Otherwise, if no input edges of the active nodes can be redirected to an unused input to increase the current level, then there are exactly $i - 1$ active function nodes and at least $D - (i - 1) \geq \alpha(n - i)$ inactive function nodes. We then estimate the probability to activate one of these nodes and leave the current level within the three first iterations of the while-loop in SAM. In the first two iterations, we make some changes to the input edges of an inactive node by assuming the worst case that these changes are required. In the last iteration, we activate the node and this ends the while-loop in SAM.

The probability to make a change to an input edge of an inactive node in the first iteration of the while-loop is at least $\frac{2 \cdot \alpha(n-i)}{3D+1}$. Then this input edge is redirected to an unused input x_u with a probability of at least

$\frac{n-i}{D+n-1} \geq \frac{n-i}{2D}$. Let j be the inactive node associated with the input edge, our argument on what may happen in the two last iterations depends on the location of j .

Subcase 2.1: Node j is located before the first (leftmost) active function node or there are no active function nodes at all. Let x_v be the input that one input edge e of the first active function node is currently connected to, or that the output connection e is currently connected to if there is no active function nodes at all. In the second iteration, the algorithm redirects the other input edge of j (the one not selected in the first iteration) to x_v , with a probability of at least $\frac{1}{3D+1} \cdot \frac{1}{D+n-1} \geq \frac{1}{(3D+1)(2D)}$. To activate j in the third iteration, it suffices to redirect edge e to j , and the probability of this event is also at least $\frac{1}{(3D+1)(2D)}$. By considering all three iterations, the overall probability to leave the current level i by SAM for this subcase is at least $\frac{2 \cdot \alpha(n-i) \cdot (n-i)}{(3D+1)^3(2D)^3} = \frac{\alpha(n-i)^2}{(3D+1)^3(4D^3)}$.

Subcase 2.2: Node j is located after the first active function node. In the second iteration, we redirect the other input edge of j to the active function node j^- that is located immediately before j , with a probability of at least $\frac{1}{(3D+1)(2D)}$. In the last iteration, we redirect the connection e (which can be the output one) that currently maintains the activeness of j^- to node j in order to activate j , with a probability of at least $\frac{1}{(3D+1)(2D)}$. The overall probability to leave the current level i for this subcase is also at least $\frac{\alpha(n-i)^2}{(3D+1)^3(4D^3)}$.

Combining all the cases gives that the probability to leave the current level i is at least $s_i := \frac{\alpha(n-i)^2}{(3D+1)^3(4D^3)}$ and summing their reciprocals over all the levels implies an expected running time of at most

$$\sum_{i=1}^{n-1} \frac{(3D+1)^3(4D^3)}{\alpha(n-i)^2} = (3D+1)^3 4D^3 \alpha^{-1} \sum_{i=1}^{n-1} \frac{1}{i^2} \leq (3D+1)^3 4D^3 \alpha^{-1} \zeta(2)$$

and this is $(3D+1)^3 D^3 ((n-1)/D)(2\pi^2/3) = \mathcal{O}(nD^5)$. The space complexity follows by applying Lemma 1 with $N_{\text{func}} = D, k = 2$ and $N_{\text{in}} = n = \mathcal{O}(D)$.

For $D \geq 10$, we have that $(3D+1)^3 \leq (3D+D/10)^3 < 30D^3$, the expected running time can be upper bounded by $20\pi^2(n-1)D^5$. Applying Lemma 10 with $p_i := \frac{\alpha(n-i)^2}{(3D+1)^3(4D^3)}$ thus $p_{\min} = \frac{\alpha}{(3D+1)^3(4D^3)} > \frac{D/(n-1)}{120D^6} = \frac{1}{120(n-1)D^5}$ and

$$\begin{aligned} \sum_{i=1}^{n-1} \frac{1}{p_i^2} &= (3D+1)^6 (16D^6) \alpha^{-2} \sum_{i=1}^{n-1} \frac{1}{(n-i)^4} \leq (30D^3)^2 (16D^6) \alpha^{-2} \sum_{i=1}^{n-1} \frac{1}{i^4} \\ &\leq 14400 \cdot D^{12} \cdot ((n-1)/D)^2 \cdot \zeta(4) = 14400 \cdot D^{10} (n-1)^2 (\pi^4/90) \\ &= 160\pi^4 (n-1)^2 D^{10} =: s, \end{aligned}$$

and with $\gamma = 20\pi^2 \delta (n-1) D^5$, implies that the probability of exceeding the running time $(1+\delta)20\pi^2(n-1)D^5$ is at most

$$\exp \left(- \min \left\{ \frac{400\pi^4 \delta^2 (n-1)^2 D^{10}}{4 \cdot 160\pi^4 (n-1)^2 D^{10}}, \frac{20\pi^2 \delta (n-1) D^5}{4 \cdot 120(n-1) D^5} \right\} \right) \leq e^{-\frac{\pi^2 \min\{\delta, \delta^2\}}{24}},$$

and since $\min\{\delta, \delta^2\} = \delta$ for $\delta \geq 1$, this is less than $e^{-\frac{\delta}{3}}$. $\square$

The bound $\mathcal{O}(n^6)$ of the minimal setting $D = n-1$ is tight and non-improvable for the (1+1) CGP* when the initial program almost already fits the training set.

Theorem 4. *There exist initial programs for which the (1+1) CGP* with $N_{\text{func}} := n-1$, and SAM mutation requires $\Omega(n^6)$ fitness evaluations in expectation to fit the complete training set of $\text{AND}_n - \{\text{AND}\} - \{x_i \mid i \in [n]\}$.*

Proof. The initial programs correspond to those obtained at the penultimate level, each having a near-optimal fitness value of 1. A concrete example is the following. The first $n-2$ function nodes are the only active ones and are connected to some $n-1$ distinct input variables. This means the program output is currently pointed to the $(n-2)$ -th function node j . Meanwhile neither the remaining free variable v nor node j is connected to the last (inactive) function node j^+ .

Modifying any input connection of the first $n - 2$ nodes is not possible since this triggers termination of the while-loop in SAM and the produced solution is rejected by the strict selection as its fitness is at least 1. Modifying the connection of j^+ without creating an optimal program is also impossible. Therefore, SAM mutation is forced to produce an optimal solution in one call by redirecting the two input connections of j^+ to j and v , then by activating j^+ (this triggers the termination of the while-loop). Such an event requires SAM to run for some $i \geq 3$ iterations in which the first $i - 1$ iterations only modify the components of node j^+ , with probability $\left(\frac{3}{3(n-1)+1}\right)^{i-1} = \left(\frac{3}{3n-2}\right)^{i-1}$, and the last iteration redirects the program output to j^+ , with probability $\frac{1}{3(n-1)+1} \cdot \frac{1}{n+(n-1)-1} = \frac{1}{(3n-2)(2n-2)}$. Conditioned on this, it still requires that among the first $i - 1$ iterations that at least one iteration redirects one input connection of j^+ to v , and another redirects the other input connection to j . The conditional probability for this by a union bound is at most $\binom{i-1}{2} \left(\frac{2}{3} \cdot \frac{1}{2n-2}\right)^2$ because two out of the three components associated with j^+ are for its input connections, and the probability of a successful redirection to a specific node is $1/(2n - 2)$.

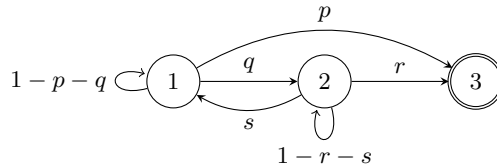
Summing the probability over all possible i gives the upper bound for the event, here we use $(i - 1)/2 \leq i - 2$ that holds for $i \geq 3$:

$$\begin{aligned}
& \sum_{i=3}^{\infty} \left(\frac{3}{3n-2}\right)^{i-1} \cdot \frac{1}{(3n-2)(2n-2)} \cdot \frac{(i-1)(i-2)}{2} \left(\frac{2}{3} \cdot \frac{1}{2n-2}\right)^2 \\
& \leq \mathcal{O}(n^{-4}) \sum_{i=3}^{\infty} (i-2)^2 \left(\frac{3}{3n-2}\right)^{i-2} \left(\frac{3}{3n-2}\right) \\
& = \mathcal{O}(n^{-5}) \sum_{i=1}^{\infty} i^2 \left(\frac{3}{3n-2}\right)^i = \mathcal{O}(n^{-5}) \cdot \text{Li}_{-2} \left(\frac{3}{3n-2}\right) \\
& = \mathcal{O}(n^{-5}) \cdot \frac{(3/(3n-2))(1 + 3/(3n-2))}{(1 - 3/(3n-2))^3} = \mathcal{O}(n^{-6}),
\end{aligned}$$

Hence, the expected time for the (1+1) CGP* to fit the complete training set is $\Omega(n^6)$. $\square$

In contrast, the upper bound for the expected running time of (1+1) CGP can be further improved by considering neutrality in the fitness landscape as follows. We need the following lemma on a three-state Markov chain. The same chain was used in [2, 28] to analyse population diversity in Genetic Algorithms.

Lemma 5. *Consider the following Markov chain with three states with the absorbing state 3 and parameters $p, q, r, s \in [0, 1]$ where $q, r > 0$.*



The expected time E_1 to reach the absorbing state 3 starting from state 1 is

$$E_1 = \frac{q + r + s}{rq + rp + sp} \leq \frac{1}{r} + \frac{1}{q} + \frac{s}{rq}.$$

Moreover, if we have $(p + q \geq q') \wedge (r \geq r') \wedge (s \leq s')$, then the above upper bound also holds with the bounding estimates, that is, $E_1 \leq \frac{1}{r'} + \frac{1}{q'} + \frac{s'}{r'q'}$.

Proof. Let E_i be the expected time to reach the absorbing state 3 starting from state i , then it is clear that $E_3 = 0$. By the law of total probability, the Markov chain satisfies the system of equations:

$$E_1 = 1 + qE_2 + (1 - p - q)E_1 \tag{1}$$

$$E_2 = 1 + sE_1 + (1 - r - s)E_2 \tag{2}$$

From (1), we get $E_2 = \frac{(p+q)E_1-1}{q} = \left(\frac{p}{q} + 1\right) E_1 - \frac{1}{q}$ and substituting in (2) gives

$$\begin{aligned} sE_1 &= (r+s) \left(\left(\frac{p}{q} + 1 \right) E_1 - \frac{1}{q} \right) - 1 \\ \Leftrightarrow \frac{r+s}{q} + 1 &= \left((r+s) \left(\frac{p}{q} + 1 \right) - s \right) E_1. \end{aligned}$$

Therefore E_1 is

$$\begin{aligned} E_1 &= \frac{\frac{r+s}{q} + 1}{(r+s) \left(\frac{p}{q} + 1 \right) - s} = \frac{\frac{(r+s)+q}{q}}{\frac{(r+s)(p+q)-sq}{q}} = \frac{q+r+s}{rq+rp+sp} \\ &= \frac{(q+r+s)/(rq)}{(rq+rp+sp)/(rq)} = \frac{\frac{1}{r} + \frac{1}{q} + \frac{s}{rq}}{1 + \frac{p}{q} + \frac{sp}{rq}} \leq \frac{1}{r} + \frac{1}{q} + \frac{s}{rq} \end{aligned}$$

as $1 + \frac{p}{q} + \frac{sp}{rq} \geq 1$ and the first upper bound statement follows. To show the second bound, we start again from E_1 but divide both the numerator and denominator by $r(p+q)$ (instead of rq in the previous calculation). This gives

$$E_1 = \frac{(q+r+s)/(r(p+q))}{(rq+rp+sp)/(r(p+q))} = \frac{\frac{q}{r(p+q)} + \frac{1}{p+q} + \frac{s}{r(p+q)}}{1 + \frac{sp}{r(p+q)}} \leq \frac{1}{r'} + \frac{1}{q'} + \frac{s'}{r'q'}$$

since $1 + \frac{sp}{r(p+q)} \geq 1$, $\frac{q}{p+q} \leq 1$, and $p+q \geq q'$, $r \geq r'$ and $s \leq s'$. $\square$

Theorem 6. *The (1+1) CGP fits the complete training set of AND_n - $\{\text{AND}\}$ - $\{x_i \mid i \in [n]\}$ using $N_{\text{func}} := D \geq n-1$, SAM mutation in $\mathcal{O}(nD^4)$ fitness evaluations in expectation.*

Proof. Similarly to the proof of Theorem 3, we divide the search space into n levels according to the number of distinct inputs used, and the (1+1) CGP is said to be at level $i \in [n]$ if its current program is connected to i distinct inputs. Again by Lemma 2 and by the elitist selection, it holds that once the algorithm has reached level i it cannot fall down to a level below i . We then estimate the expected time τ_i to leave the current level i to a higher level.

To estimate of τ_i more precisely, by considering non-strict improvement steps as this is allowed by the (1+1) CGP, we partition each level i into two sublevels i^+ and i^- which match exactly Cases 1 and 2 in the previous proof. That is, the algorithm is said to be at level i^+ (Case 1) if it has at least one input edge of an active node that can be redirected to an unused input, and it is at level i^- (Case 2) otherwise. We then apply Lemma 5 with states 1, 2, 3 being levels i^- , i^+ , $i+1$ respectively, and τ_i is bounded from above by E_1 of the lemma.

It remains to bound the parameters $p(i)+q(i), r(i), s(i)$ so that we can deduce an upper bound on $E_1(i)$. It follows from the same argument as in Case 1 of the previous proof that $r(i) \geq \frac{n-i}{(3D+1)(2D)} =: r'(i)$.

To bound $p(i)+q(i)$ from below, we follow Case 2, its subcases and use the same notation per subcase and the same assumption but make slightly different changes within only two iterations of SAM mutation because we only need to reach level i^+ (and not necessarily $i+1$) from i^- . In Subcase 2.1, the probability of selecting an input edge of an active function node j for change in the first iteration is still at least $\frac{2\alpha(n-i)}{3D+1}$, but then it suffices to redirect that edge to x_v thus with probability $\frac{1}{D+n-1} \geq \frac{1}{2D}$, then in the second iteration to redirect edge e (either of the output or of the first active function node) to j , with a probability of at least $\frac{1}{(3D+1)(2D)}$. In Subcase 2.2, we redirect the selected input edge of the function node j to the first active node j^- before j in the first iteration with probability $\frac{1}{D+n-1} \geq \frac{1}{2D}$, then redirect edge e to j in the second iteration with a probability of at least $\frac{1}{(3D+1)(2D)}$. Therefore, we have that the probability of transiting from state i^- to at least i^+ is $p(i)+q(i) \geq \frac{2\alpha(n-i)}{(3D+1)^2(2D)^2} =: q'(i)$.

To bound $s(i)$ from above, we note that for the algorithm to be in level i^+ its current program must either have at least one input with multiple input connections from active function nodes to it, or have at least one

active function node whose activeness is maintained by multiple input connections from other active function nodes. In the former case, for an upper bound argument on $s(i)$, we only need to focus one such input and two specific connections from the active nodes to the input because to return to level i^- at least one of the connections must no longer come from an active node. In other words, there are at most two active nodes that can be deactivated/changed to return to level i^- , and this happens with a probability of at most $2 \cdot \frac{1}{3D+1}$ by one SAM mutation. The argument is similar for the later case as again we only need to focus on one active function node and two specific connections that maintain the node activeness. Therefore, we have $s(i) \leq \frac{2}{3D+1} =: s'(i)$.

Applying Lemme 5 and summing all τ_i give the expected running time:

$$\begin{aligned} \sum_{i=1}^{n-1} \tau_i &\leq \sum_{i=1}^{n-1} \frac{1}{r'(i)} + \frac{1}{q'(i)} + \frac{s'(i)}{r'(i)q'(i)} \\ &= ((3D+1)(2D) + (3D+1)^2(2D)^2(2\alpha)^{-1}) \sum_{i=1}^{n-1} \frac{1}{n-i} \\ &\quad + 2(3D+1)^2(2D)^3(2\alpha)^{-1} \sum_{i=1}^{n-1} \frac{1}{(n-i)^2} \\ &= \mathcal{O}(nD^3) H_{n-1} + \mathcal{O}(nD^4)\zeta(2) = \mathcal{O}(nD^4). \end{aligned}$$

□

The analyses from Theorems 3 and 6 show that using more than the minimum $n-1$ function nodes can benefit CGP by increasing the bounds s_i in the harder case, and also $q'(i)$ for $p(i) + q(i)$ in the Markov chain.

4 Analysis of Evolving Exclusive Disjunctions

The second target function that we consider is the XOR_n function of $n \geq 2$ inputs where $\text{XOR}_n(x_1, \dots, x_n) := x_1 \oplus x_2 \oplus \dots \oplus x_n$ that must be built from at least $n-1$ binary XOR gates. This function is of interest for us because it is not evolvable [38] even though it is PAC-learnable [37] thus a candidate problem to show a negative result for CGP, especially for an advanced operator like SAM. We adopt the name $\text{XOR}_n\text{-}\{\text{XOR}\}\text{-}\{x_i \mid i \in [n]\}$ for this problem. A candidate CGP program is a function whose *raw form* may contain an input variable repeated multiple times, e.g. $x_1 \oplus x_1 \oplus x_2$ where x_1 is repeated twice. However, a variable that is repeated an even number of times is as if it is ignored, and it is repeated an odd number of times is as if it is repeated once. We refer to the function form after applying these simplifications as the *simplified form*.

For the $\text{XOR}_n\text{-}\{\text{XOR}\}\text{-}\{x_i \mid i \in [n]\}$ and using the complete training set $\mathcal{K}_{\text{XOR}_n}$, the fitness of a candidate program y can only take two possible values, that is, $f(y) = 0$ if the simplified form of y is exactly XOR_n , i.e. every input appears an odd number of times in the raw form, otherwise $f(y) = 2^{n-1}$, i.e. y is incorrect on a half of the rows (and correct on the other half) [22]. This property implies that if the use of an incomplete training set leads to a program that ignores some inputs in the simplified form, then the generalisation ability of such a program is always $2^{n-1}/2^n = 1/2$. This is as bad as a random guess, and justifies our focus on the setting of the complete training set $S = \mathcal{K}_{\text{XOR}_n}$.

We then expect CGP to perform a random walk in the space of programs until it realises an optimal program, i.e. having the simplified form XOR_n . To show an exponential time on reaching such a program, we first show that it is unlikely that such a program can emerge from a fixed subset L of function nodes. For a lower bound argument, it suffices to focus on two necessary properties of an optimal program built out of L : (i) every input is connected to L by an odd number of connections; (ii) the output of at least $n-2$ nodes of L is connected to other nodes in L . The second property is required because an optimal program has at least $n-1$ active function nodes, thus at least $n-2$ of those have their activeness maintained by L . Formally, an input variable x_i is called an *odd variable* with respect to L if it is connected to L by an odd number of connections, otherwise it is an *even variable* (with respect to L , we often omit this part when L is clear from the context). Thus we track the number

of odd variables, denoted $V(L)$, and $U(L) := |\{\ell \in L \mid \text{the output of } \ell \text{ is connected to another node in } L\}|$ of the current program in CGP to reach their respective values n and $n - 2$.

Lemma 7. *A randomly generated CGP program on $\text{XOR}_n\text{-}\{\text{XOR}\}\text{-}\{x_i \mid i \in [n]\}$ using $N_{\text{func}} := D \geq n - 1$ has $V(L) \leq 3n/4$ with probability $1 - e^{-\Omega(n)}$, for any fixed set L of function nodes with $m := |L| \in \text{poly}(n)$.*

Proof. We are only interested in the sampling steps where the $2m$ connections of L are sampled to connect to an input variable because those are the only steps that can change $V(L)$. Let V_j be $V(L)$ of the sampled program after the j -th step and we apply the negative drift theorem (Theorem 11) with potential $X_j := n - V_j$. Note that $X_0 = n$ as no connections of L have been sampled yet and zero is an even number, and that $|X_{j+1} - X_j| = 1$ as the sampling toggles the parity of exactly one input variable.

Each considered sampling step chooses a input variable $x_1, \dots, x_n$ uniformly at random. Thus for $X_j \in [n/4, 3n/8]$, there are at least $V_j = n - X_j \geq n - 3n/8 = 5n/8$ odd variables, which increase X_j if one is chosen. Otherwise if an even variable is chosen then X_j is decreased. Thus, we have a drift

$$\mathbb{E}[X_{j+1} - X_j \mid X_j \in [n/4, 3n/8]] \geq 5/8 - 3/8 = 1/4.$$

Applying Theorem 11 with parameters $a = n/4$, $b = 3n/8$, $r(\ell) = 1$, and $\delta = 1/4$, we obtain for $T = \min\{j \in \mathbb{N} \mid X_j \leq n/4\}$ that $P(T \leq e^{\varepsilon n}) \leq e^{-\Omega(n)}$ for a suitable constant $\varepsilon > 0$. Since $T > 2m$ implies the final $V(L)$ of the initialisation is below $3n/4$, thus event $\{T > 2m\} \subseteq \{\text{final } V(L) < 3n/4\}$ and we get $\Pr(\text{final } V(L) < 3n/4) \geq \Pr(T > 2m) = 1 - \Pr(T \leq 2m) \geq 1 - \Pr(T \leq e^{\varepsilon n}) \geq 1 - e^{-\Omega(n)}$ as $2m \leq e^{\varepsilon n}$ for sufficiently large n . $\square$

If an optimal program has not yet been found, then the (1+1) CGP always accepts offspring programs and all modifications by every iteration of the while-loop across SAM mutations are taken into account. We refer to each of these iterations as an *elementary step*. The following lemma, again using the negative drift theorem, shows that it is unlikely for a program with the two necessary properties, let alone for an optimal program, to emerge during the application of an exponential number of elementary steps to a randomly generated program.

Lemma 8. *Starting from a randomly generated CGP program on $\text{XOR}_n\text{-}\{\text{XOR}\}\text{-}\{x_i \mid i \in [n]\}$ using $N_{\text{func}} := D \geq n - 1$, for a fixed set L of its function nodes with $m := |L| \leq (1 + d)n$ for any $d \leq 10^{-2}$, the probability that a program with $U(L) \geq n - 2$ and $V(L) = n$ emerges during the application of $e^{\varepsilon n}$ elementary steps of SAM for some constant $\varepsilon > 0$ is at most $e^{-\Omega(n)}$.*

Proof. For $m < n - 1$, the result is trivial as $V(L)$ cannot reach n , so we assume the opposite. Let U_j and V_j be $\min(U(L), n - 2)$ and $V(L)$ respectively after applying j elementary steps. We can also neglect all steps where U_j and V_j are unchanged as including those can only increase the number of required steps.

We apply the negative drift theorem with potential $Y_j := 2n - 2 - U_j - V_j$. In any single iteration, the configuration of at least $n - 2$ input variables remains unchanged, so $|V_{j+1} - V_j| \leq 2$. Since only one link is modified per iteration, it is clear that $|U_{j+1} - U_j| \leq 1$. Therefore, $|Y_{j+1} - Y_j| \leq 3$ by triangle inequality. Further, V_j is bounded from above by n , and U_j is by $n - 2$.

It follows from Lemma 7, that $V_0 \leq 3n/4$ with probability $1 - e^{-\Omega(n)}$. Assuming this occurs then $Y_0 \geq 2n - 2 - (n - 2) - 3n/4 \geq n/4$, and we estimate the drift $\mathbb{E}[Y_{j+1} - Y_j \mid Y_j]$ in the interval $Y_j \in [n/256 - 2, n/128 - 2]$.

In this interval it holds that

$$U_j = 2n - 2 - Y_j - V_j \geq 2n - 2 - \left(\frac{n}{128} - 2\right) - n = \frac{127n}{128} \quad (3)$$

$$V_j = 2n - 2 - Y_j - U_j \geq 2n - 2 - \left(\frac{n}{128} - 2\right) - (n - 2) \geq \frac{127n}{128} + 2 \quad (4)$$

To increase Y_j , it suffices to select a connection pointing to an odd variable and reconnect it to another odd variable (w. r. t. L), i. e. with probability at least

$$\frac{V_j}{2m} \cdot \frac{V_j - 1}{n + m - 1} \stackrel{(4)}{>} \frac{(127n/128)^2}{(2m)^2} > \frac{6n^2}{25D^2} =: p_+.$$

for n sufficiently large. This increases Y_t by two. There are also other possibilities to increase Y_t but we assume they only contribute zero to the drift.

To decrease Y_j , it is necessary that one of the following two events happens. Note that some subsequent change following those events may lead to eventually no change in Y_j , but we pessimistically assume that Y_j always is decreased by 3.

(A) Select a connection pointing to an even variable (then reconnect it).

(B) Select a connection not pointing to an input variable and either reconnect it to an even variable, or reconnect it to a function node of L which is not currently counted in $U(L)$.

There are at most $2D - U_j - V_j \leq 2n + 2dn - U_j - V_j = 2dn + 2 + Y_j \leq n/128 + 2dn$ connections pointing to an even variable, thus $\Pr(A) \leq (n/128 + 2dn)/(2m) \leq 1/128 + 2d$. Here we use $n/(2m) \leq n/(2(n-1)) \leq 1$ for $n \geq 2$.

It follows from (3) that at most $D - 1 - U_j \leq n + dn - 1 - 127n/128 < n/128 + dn$ function nodes have their output not used within L , and from (4) that there are at most $n - V_j \leq n - (127n/128 - 2) < n/128$ even variables. Note that there are at least n choices for reconnection, so $\Pr(B) \leq (n/128 + dn)/n + n/(128n) \leq 1/64 + d$. Here we assume that the probability of selecting the right connection type is at most 1.

So one of the two events happens with probability at most

$$\Pr(A) + \Pr(B) \leq \frac{1}{128} + 2d + \frac{1}{64} + d < \frac{1}{42} + 3d =: p_-.$$

Since Y_j can only decrease by at most three in each iteration, we obtain for the drift for n sufficiently large and any $d \leq 10^{-2}$ that

$$\begin{aligned} \mathbb{E}[Y_{j+1} - Y_j \mid Y_j \in [\frac{n}{256} - 2, \frac{n}{128} - 2]] &> 2 \cdot p_+ - 3 \cdot p_- = \frac{12n^2}{25D^2} - \frac{3}{42} - 9d \\ &= \frac{n}{D} \left(\frac{12n}{25D} - \frac{3D}{42n} - \frac{9Dd}{n} \right) \geq \frac{1}{1+d} \left(\frac{12}{25(1+d)} - \frac{3(1+d)}{42} - 9(1+d)d \right) \\ &> \frac{1}{1+10^{-2}} \left(\frac{12}{25(1+10^{-2})} - \frac{3(1+10^{-2})}{42} - 9(1+10^{-2})10^{-2} \right) > \frac{3}{10} =: \delta. \end{aligned}$$

Applying Theorem 11 with parameters $a = n/256 - 2$, $b = n/128 - 2$, $r(\ell) = 3$ and δ we obtain for $T := \min\{j \in \mathbb{N} \mid Y_j \leq a\}$ that $P(T \leq e^{-\varepsilon n}) \leq e^{-\Omega(n)}$ for a suitable constant $d > 0$. Hence, even including the event at initialisation, with probability at least $1 - e^{-\Omega(n)}$ for n sufficiently large, we still have $Y_j > n/256$ and thus $U_j + V_j < 2n - 2$, or equivalently $(V_j < n) \vee (U_j < n - 2)$ throughout the application of $e^{\varepsilon n}$ elementary steps. $\square$

To extend the lemma to (1+1) CGP, we take a union bound over all subsets L , whose number can be estimated via the approximation of the standard binomial coefficient bounds using binary entropy. Finally, the SAM while-loop terminates once the output connection is selected for modification, which occurs with probability $1/(3D + 1) = \Omega(1/n)$ for $D = \mathcal{O}(n)$. Hence, an exponential number of elementary SAM steps translates to the same order of SAM mutations (fitness evaluations) by a Chernoff bound.

Theorem 9. *Let $d < 10^{-2}$ be a sufficiently small constant. The probability that the (1+1) CGP fails to fit the complete training set of $\text{XOR}_n - \{\text{XOR}\} - \{x_i \mid i \in [n]\}$ within $e^{\Omega(n)}$ fitness evaluations using SAM mutation and $N_{\text{func}} := D$ for $n - 1 \leq D \leq n + dn$, is $1 - e^{-\Omega(n)}$.*

Proof. It follows from Lemma 8 that the probability that an optimal program emerges from a fixed subset L of m function nodes with $m \in [n - 1, n + dn]$ is at most $e^{-\Omega(n)} \leq e^{-\kappa n}$ after applying $e^{\varepsilon n}$ elementary steps of SAM to a randomly generated program for some constants $\kappa > 0$ and $\varepsilon > 0$. By a union bound over all possible such subsets L , we get a required running time for the (1+1) CGP in terms of number of elementary steps. Since we

AND _n param.	Setting	GP param.	Setting
Dimension n	$\{3, \dots, 15\}$ (compl.), $\{3, \dots, 50\}$ (incompl.)	Maximum iterations	1×10^6
Input i	$i \in \{0, 1\}$	Search strategy	(1+1)
Function set	$F = \{\text{AND}\}$	Selection types	non-strict, strict
Terminal set	$T = \{x_1 \dots x_n\}$	No. replications per n	30

Table 1: Configuration of GP and AND_n for the experiments.

may assume that $D \leq n + dn \leq 2n - 2$, we may estimate the number of such subsets by

$$\sum_{i=n-1}^D \binom{D}{i} \leq (D-n) \binom{D}{n-1} \leq (D-n) 2^{D \cdot H((n-1)/D)}$$

for n sufficiently large where $H(\alpha) = -\alpha \log \alpha - (1-\alpha) \log(1-\alpha)$ for $\alpha \in (0, 1)$ denotes the binary entropy function. The last inequality uses $\binom{n}{k} \leq 2^{nH(k/n)}$, which holds for all $k \in [n]$ (see for example [3]). Note that $H(\alpha)$ is strictly monotone decreasing in $\alpha \in [1/2, 1)$, and that $(n-1)/D \geq 1/(1+d+o(1))$ and hence, $D \cdot H((n-1)/D) \leq n \cdot (1+d) \cdot H(1/(1+d+o(1))) < n\kappa' < n\kappa$ for a suitable constant $\kappa' > 0$, thus the required running time is still exponential.

To count this time in terms of fitness evaluations, we note that for a SAM mutation, its while-loop ends if the output node is chosen for modification, i.e. with probability at least $1/(3D+1) = \Omega(1/n)$. Now consider a sum X of e^{dn} Bernoulli random variables, each becoming one if the output is selected for modification and zero otherwise. Then because a SAM mutation can also be stopped by modifying an active function node, X is a lower bound for the number of fitness evaluations. By a classical Chernoff bound, we have that $\Pr(X \geq e^{cn}) = 1 - e^{-\Omega(n)}$ for a sufficiently small constant $c > 0$, and this concludes our proof. $\square$

5 Experiments

We evaluate CGP and TGP on AND_n through two sets of experiments. First, using the complete training set, we measure runtime to highlight the performance gap between the two approaches when evolving the exact solution. Second, with incomplete training and testing sets, we assess both runtime and generalisation ability to examine their generalisation performance.

Experimental setup: We used TinyverseGP [13], a modular benchmarking Python library for GP that provides modules for various flavours of GP including CGP and TGP, to perform the experiments. Using the existing implementations `TinyTGP` and `TinyCGP` of the library, we derived the models `SimpleTGP` and `SimpleCGP` which correspond to (1 + 1) TGP/CGP as described in Section 2.1 and [16, 15]. The CGP model uses `MUTATIONSAM` as described in Algorithm 3, while the TGP model uses the mutations from [15] which are based on the HVL-prime operator [27]. The implementations of the models and the AND_n as well as the raw data obtained in the experiments are provided in our repository¹.

We evaluated various settings of n in each scenario. For the scenario where the complete training set is used, we evaluated $n \in \{3, \dots, 15\}$. However, since the evaluation of an incomplete training set is less expensive, we evaluated the performance for larger settings of $n \in \{3, \dots, 50\}$ in this scenario. For each value of n , 30 runs were performed with different random seeds per model (CGP and TGP). The mean of each sample per n was then used for the illustration of our results. The GP configuration used in our experiments is summarised in Table 1. Running time was measured as the number of iterations for the algorithm to fit the training set. In our plots, we refer to the TGP models as `simple_tgp_single` and `simple_tgp_multi` and to the CGP model as `simple_cgp_sam`. For CGP, we defined the length of the genotype N_{func} with respect to the dimension n but also considered extra nodes to see if there is any gap in runtime. More precisely, we used $N_{\text{func}} \in \{n, 2n, 3n\}$, and refer to as **1n**, **2n**, **3n** in our results.

¹<https://github.com/GPBench/TinyverseGP/tree/ppsn-2026>

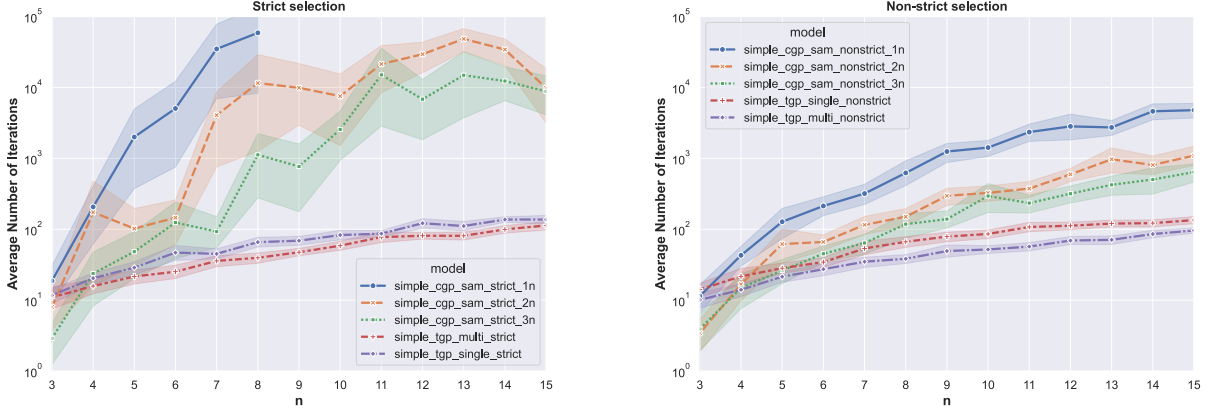


Figure 2: Results of the comparison on AND_n using complete datasets.

We set the size of the incomplete training and testing sets S and S' as $|S| = |S'| = \lceil n^k \rceil$ with $k = 1.3$ for the settings of large n . Both sets were generated uniformly at random. Approximating the generalisation ability $G_g(y)$ by $\hat{G}(y, S')$ was done with the formula as explained in Section 2.2. Statistics are reported per setting of n excluding unsuccessful runs, i.e. exceeding the maximum number of iterations without fitting the training set.

Results: Figure 2 shows the results for the running time comparison on AND_n when the complete training set was used. It can be clearly seen that the considered TGP models perform generally better in this scenario. It can also be seen that increasing the number of function nodes generally improves the search performance of CGP while using strict selection deteriorates its performance. Figure 3 show the results of our experiments with incomplete training and testing sets and it can also be seen that increasing the number of function nodes improves the search performance of CGP in terms of running time for both selection strategies. It is also evident that the generalisation error is relatively low for all models considered demonstrating good generalisation abilities on AND_n . Detailed statistical evaluation of the results can be found in the appendix document.

6 Conclusions and Discussions

We presented the first runtime analysis of CGP for evolving conjunctions and disjunctions, considering both complete and incomplete training sets. For AND_n with n variables and D nodes, we proved expected runtimes of $O(nD^5)$ for (1+1) CGP* and $O(nD^4)$ for (1+1) CGP with the complete training set. Our analysis revealed the characteristics of CGP induced search that enabling the acceptance of equally good solutions with active gates non-contributing to fitness is beneficial. We also showed that (1+1) CGP fails with overwhelming probability on the complete training set of the XOR_n . To complement our theoretical analysis, we conducted an empirical study examining the runtime performance and generalisation ability of CGP and TGP models for AND_n .

The theoretical analysis of incomplete training sets for XOR_n is left out for the reason that the generalisation ability one can hope for is only as good as a random guess. For AND_n , there is a technical obstacle that on an incomplete training set, our approach using the fitness-level method and Markov chain analysis no longer works with the current level partition. In particular, it is possible to swap a larger set of input variables for a smaller one, and such a change still improves the fitness. The same issue must occur for TGP under a global mutation operator, however we could not find any literature on this. We also believe drift analysis [6] is a better-suited tool for addressing such a setting, and leave the analysis for future work. Future work should also include extending the analysis to general Boolean formulas with AND/OR, identifying broader function classes where CGP is effective, and studying its behaviour on flat fitness landscapes. Achieving these, however, requires a deeper understanding of the underlying dynamics of CGP, which still is largely unexplored.

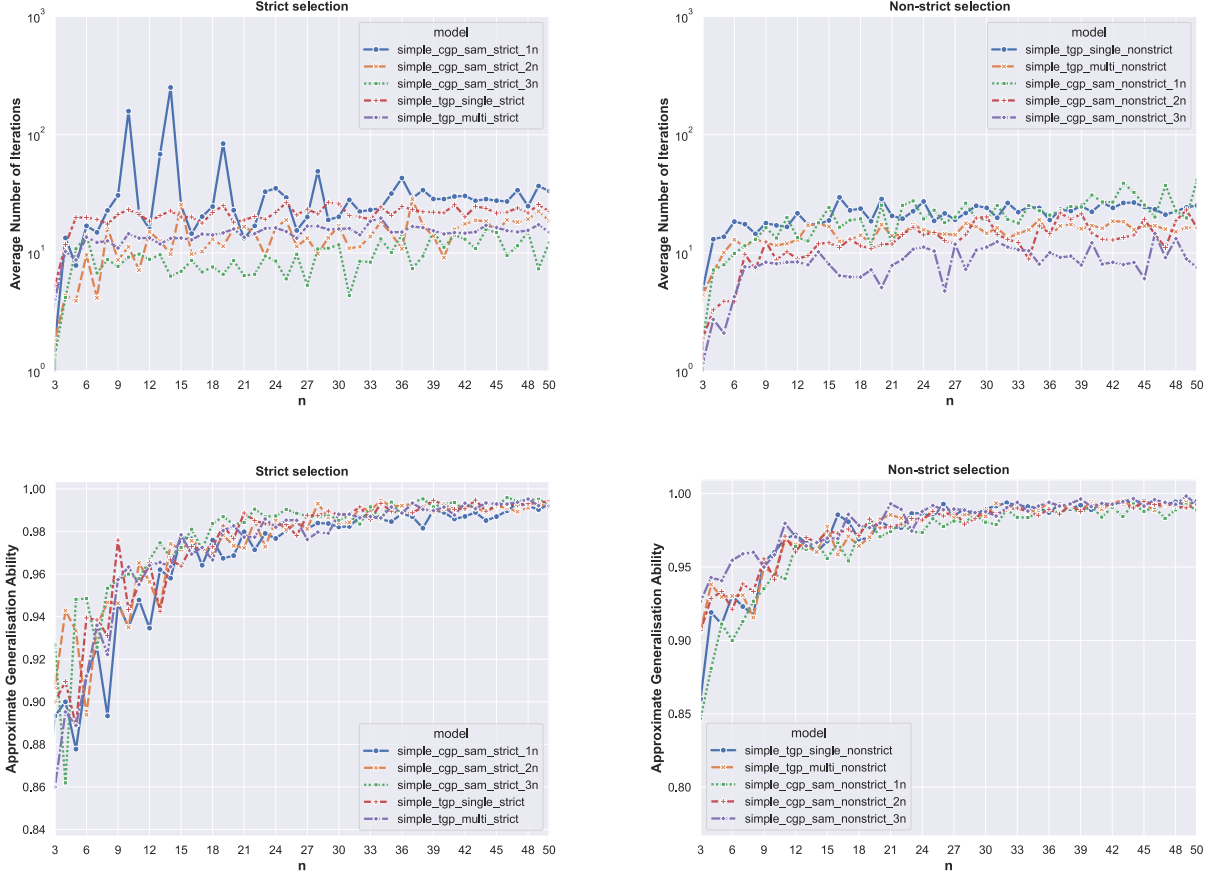


Figure 3: Results of the comparison on AND_n using incomplete datasets.

References

- [1] Cormen, T.H., Leiserson, C.E., Rivest, R.L., Stein, C.: Introduction to Algorithms. MIT Press (2009)
- [2] Corus, D., Oliveto, P.S.: Standard steady state genetic algorithms can hillclimb faster than mutation-only evolutionary algorithms. *IEEE Transactions on Evolutionary Computation* **22**(5), 720–732 (2018)
- [3] Cover, T.M., Thomas, J.A.: Elements of Information Theory. Wiley (2006)
- [4] Cramer, N.L.: A representation for the adaptive generation of simple sequential programs. In: *Proceedings of the 1st International Conference on Genetic Algorithms*. pp. 183–187. Lawrence Erlbaum Associates (1985)
- [5] Dickmanns, D., Schmidhuber, J., Winklhofer, A.: Der genetische algorithmus: Eine implementierung in prolog. *Fortgeschrittenenpraktikum, Institut für Informatik, Lehrstuhl Prof. Radig, Technische Universität München* (1987)
- [6] Doerr, B., Johannsen, D., Winzen, C.: Multiplicative drift analysis. In: *GECCO '10: Proceedings of the 12th annual conference on Genetic and Evolutionary computation*. p. 1449–1456. ACM (2010)
- [7] Doerr, B., Lissovoi, A., Oliveto, P.S.: Evolving boolean functions with conjunctions and disjunctions via genetic programming. In: *GECCO '19: Proceedings of the Genetic and Evolutionary Computation Conference*. p. 1003–1011. ACM (2019)
- [8] Doerr, B., Lissovoi, A., Oliveto, P.S.: (1+1) genetic programming with functionally complete instruction sets can evolve boolean conjunctions and disjunctions with arbitrarily small error. *Artificial Intelligence* **319**, 103906 (2023)

- [9] Goldman, B.W., Punch, W.F.: Reducing wasted evaluations in cartesian genetic programming. In: EuroGP'13: Proceedings of the 16th European conference on Genetic Programming. Lecture Notes in Computer Science, vol. 7831, pp. 61–72. Springer (2013)
- [10] Hachtel, G.D., Somenzi, F.: Logic synthesis and verification algorithms. Kluwer (1996)
- [11] Husa, J., Sekanina, L.: Evolving cryptographic boolean functions with minimal multiplicative complexity. In: IEEE Congress on Evolutionary Computation, CEC 2020. IEEE (2020)
- [12] Kalkreuth, R., Droschinsky, A.: On the time complexity of simple cartesian genetic programming. In: IJCCI 2019: Proceedings of the 11th International Joint Conference on Computational Intelligence. pp. 172–179. ScitePress (2019)
- [13] Kalkreuth, R., de França, F.O., Jankovic, A., Anastacio, M., Dierkes, J., Vasicek, Z., Hoos, H.: Tiny-versegp: Towards a modular cross-domain benchmarking framework for genetic programming. In: GECCO '25 Companion: Proceedings of the Genetic and Evolutionary Computation Conference Companion. p. 127–130. ACM (2025)
- [14] Kalkreuth, R., Vašíček, Z., Husa, J., Vermetten, D., Ye, F., Bäck, T.: General boolean function benchmark suite. In: Proceedings of the 17th ACM/SIGEVO Conference on Foundations of Genetic Algorithms. p. 84–95 (2023)
- [15] Kötzing, T., Sutton, A.M., Neumann, F., O'Reilly, U.M.: The max problem revisited: The importance of mutation in genetic programming. In: GECCO '12: Proceedings of the 14th annual conference on Genetic and evolutionary computation. p. 1333–1340. ACM (2012)
- [16] Kötzing, T., Sutton, A.M., Neumann, F., O'Reilly, U.M.: The max problem revisited: The importance of mutation in genetic programming. Theoretical Computer Science **545**, 94–107 (2014)
- [17] Koza, J.R.: Hierarchical genetic algorithms operating on populations of computer programs. In: Proceedings of the 11th international joint conference on Artificial intelligence - Volume 1. pp. 768–774. Morgan Kaufmann (1989)
- [18] Koza, J.R.: Genetic Programming: On the Programming of Computers by Means of Natural Selection. MIT Press (1993)
- [19] Lissovoi, A., Oliveto, P.S.: On the time and space complexity of genetic programming for evolving boolean conjunctions. In: Proceedings of the Thirty-Second AAAI Conference on Artificial Intelligence. AAAI Press (2018)
- [20] Lissovoi, A., Oliveto, P.S.: On the time and space complexity of genetic programming for evolving boolean conjunctions. Journal of Artificial Intelligence Resesearch **66**, 655–689 (2019)
- [21] Lissovoi, A., Oliveto, P.S.: Computational complexity analysis of genetic programming. In: Theory of Evolutionary Computation - Recent Developments in Discrete Optimization, pp. 475–518. Natural Computing Series, Springer, Cham, Switzerland (2020)
- [22] Mambrini, A., Oliveto, P.S.: On the analysis of simple genetic programming for evolving boolean functions. In: Proceedings of the 19th European Conference on Genetic Programming, EuroGP 2016. pp. 99–114. Springer (2016)
- [23] Miller, J.F.: An empirical study of the efficiency of learning boolean functions using a cartesian genetic programming approach. In: GECCO'99: Proceedings of the 1st Annual Conference on Genetic and Evolutionary Computation - Volume 2. vol. 2, pp. 1135–1142. Orlando, Florida, USA (1999)

- [24] Miller, J.F., Smith, S.L.: Redundancy and computational efficiency in cartesian genetic programming. *IEEE Transactions on Evolutionary Computation* **10**(2), 167–174 (2006)
- [25] Miller, J.F., Thomson, P.: Cartesian genetic programming. In: *Genetic Programming: European Conference, EuroGP 2000*. pp. 121–132. Springer Berlin Heidelberg, Berlin, Heidelberg (2000)
- [26] Miller, J.F.: An empirical study of the efficiency of learning boolean functions using a cartesian genetic programming approach. In: *Proceedings of the 1st Annual Conference on Genetic and Evolutionary Computation - Volume 2*. p. 1135–1142. Morgan Kaufmann (1999)
- [27] Neumann, F., O’Reilly, U.M., Wagner, M.: Computational complexity analysis of genetic programming - initial results and future directions. In: *Genetic Programming Theory and Practice IX*, pp. 113–128. Springer New York (2011)
- [28] Oliveto, P.S.: Time complexity analysis of stochastic search algorithms. In: *Handbook of Heuristics (Second Edition)*, pp. 849–884. Springer, Cham, Switzerland (2025)
- [29] Oliveto, P.S., Witt, C.: Simplified drift analysis for proving lower bounds in evolutionary computation. *Algorithmica* **59**(3), 369–386 (2011)
- [30] Oliveto, P.S., Witt, C.: Erratum: Simplified drift analysis for proving lower bounds in evolutionary computation. *CoRR* **abs/1211.7184** (2012)
- [31] Openshaw, S., Turton, I.: Building new spatial interaction models using genetic programming. In: *Evolutionary Computing, , AISB workshop*. pp. 11–13. Springer Berlin Heidelberg (1994)
- [32] Perkis, T.: Stack-based genetic programming. In: *Proceedings of the First IEEE Conference on Evolutionary Computation*. pp. 148–153. IEEE Press (1994)
- [33] Poli, R.: Parallel distributed genetic programming. Technical Report CSRP-96-15, School of Computer Science, University of Birmingham (1996)
- [34] Ryan, C., Collins, J.J., O’Neill, M.: Grammatical evolution: Evolving programs for an arbitrary language. In: *Proceedings of the First European Workshop on Genetic Programming*. pp. 83–96. Springer (1998)
- [35] Turner, A.J., Miller, J.F.: Cartesian genetic programming: Why no bloat? In: *EuroGP 2014: Revised Selected Papers of the 17th European Conference on Genetic Programming*. pp. 222–233. Springer (2014)
- [36] Turner, A.J., Miller, J.F.: Neutral genetic drift: an investigation using cartesian genetic programming. *Genetic Programming and Evolvable Machines* **16**(4), 531–558 (2015)
- [37] Valiant, L.G.: A theory of the learnable. In: *STOC’84: Proceedings of the sixteenth annual ACM symposium on Theory of Computing*. p. 436–445. ACM (1984)
- [38] Valiant, L.G.: Evolvability. *Journal of ACM* **56**(1), 3:1–3:21 (2009)
- [39] Vasíček, Z.: Cartesian GP in optimization of combinational circuits with hundreds of inputs and thousands of gates. In: *Genetic Programming - 18th European Conference, EuroGP 2015, Copenhagen, Denmark, April 8-10, 2015, Proceedings*. pp. 139–150. Springer (2015)
- [40] Vasíček, Z., Sekanina, L.: Evolutionary approach to approximate digital circuits design. *IEEE Transactions on Evolutionary Computation* **19**(3), 432–444 (2015)
- [41] Vasíček, Z., Sekanina, L.: Evolutionary design of complex approximate combinational circuits. *Genetic Programming and Evolvable Machines* **17**(2), 169–192 (2016)
- [42] Witt, C.: Fitness levels with tail bounds for the analysis of randomized search heuristics. *Information Processing Letters* **114**(1-2), 38–41 (2014)

- [43] Yu, T., Miller, J.: Neutrality and the evolvability of boolean function landscape. In: Genetic Programming. pp. 204–217. Springer Berlin Heidelberg (2001)

A Useful Tools for the Analysis

The following tail bounds for sums of geometric random variables are from [42].

Lemma 10 (Theorem 1 in [42]). *Let $(X_i)_{i \in [n]}$ be n independent random variables following geometric distributions with the corresponding success probabilities $(p_i)_{i \in [n]}$, and define $X := \sum_{i=1}^n X_i$, $p_{\min} := \min\{p_i \mid i \in [n]\}$. If $s \geq \sum_{i=1}^n (1/p_i^2)$ for some $s < \infty$ then*

$$\Pr(X \geq \mathbb{E}[X] + \gamma) \leq e^{-\frac{\gamma}{4} \cdot \min\{\frac{\gamma}{s}, p_{\min}\}} \text{ and } \Pr(X \leq \mathbb{E}[X] - \gamma) \leq e^{-\frac{\gamma^2}{2s}}.$$

The negative drift theorem is from [29, 30].

Theorem 11 (Theorem 2 in [30]). *Let $(X_t)_{t \in \mathbb{N}}$ be a real-value stochastic process over some state space. Suppose there exist an interval $[a, b] \subset \mathbb{R}$, two constants $\delta, \varepsilon > 0$, and possibly depending on $\ell := b - a$ a function $r(\ell)$ such that $1 \leq r(\ell) = o(\ell / \log \ell)$ such that for all $t \geq 0$ the following conditions hold:*

1. $\mathbb{E}[X_{t+1} - X_t \mid X_t, a < X_t < b] \geq \varepsilon$,
2. $\Pr(|X_{t+1} - X_t| \geq j \mid X_t, a < X_t) \leq \frac{r(\ell)}{(1+\delta)^j}$ for $j \in \mathbb{N}$.

Then there exists a constant $c > 0$ such that for $T := \min\{t \geq 0 \mid X_t \leq a\}$ it holds that $\Pr(T \leq 2^{c\ell/r(\ell)} \mid X_0 \geq b) = 2^{-\Omega(\ell/r(\ell))}$.

B Additional Statistical Evaluation of the Experimental Results

Table 2 shows the statistical evaluation for the comparison of running time between TGP and CGP on AND_n using non-strict and strict selection as well as complete datasets.

selection	n	Model	Successful Runs	Avg. Iterations	SD	Min	Q1	Median	Q3	Max
strict	3	simple_egg_sam_strict_1n	30	19	32	0	3	10	17	167
		simple_egg_sam_strict_2n	30	8	14	0	0	2	8	61
		simple_egg_sam_strict_3n	30	3	5	0	0	0	4	18
		simple_tgp_multi_strict	30	11	12	2	4	8	14	60
		simple_tgp_single_strict	30	12	8	2	6	8	18	30
	4	simple_egg_sam_strict_1n	29	206	483	2	11	29	85	1985
		simple_egg_sam_strict_2n	30	174	807	0	2	12	25	4439
		simple_egg_sam_strict_3n	30	24	57	0	1	6	14	304
		simple_tgp_multi_strict	30	16	10	1	8	15	22	47
		simple_tgp_single_strict	30	20	10	7	12	18	25	47
	5	simple_egg_sam_strict_1n	29	2002	7359	1	35	198	833	39099
		simple_egg_sam_strict_2n	30	102	238	4	12	18	50	1062
		simple_egg_sam_strict_3n	30	48	109	0	10	20	33	595
		simple_tgp_multi_strict	30	22	14	5	8	20	33	54
		simple_tgp_single_strict	30	29	18	5	16	24	38	80
	6	simple_egg_sam_strict_1n	20	5055	14315	13	38	224	2418	63331
		simple_egg_sam_strict_2n	30	146	287	2	21	50	103	1396
		simple_egg_sam_strict_3n	30	124	300	0	12	31	96	1322
		simple_tgp_multi_strict	30	25	15	8	17	24	28	83
		simple_tgp_single_strict	30	47	32	8	27	42	58	168
	7	simple_egg_sam_strict_1n	15	34906	80917	33	304	7568	22293	314957
		simple_egg_sam_strict_2n	30	4061	11429	10	83	174	1054	51464
		simple_egg_sam_strict_3n	30	93	132	20	30	58	97	715
		simple_tgp_multi_strict	30	36	17	10	24	28	50	77
simple_tgp_single_strict		30	45	21	16	29	44	59	97	
8	simple_egg_sam_strict_1n	17	58849	138929	134	344	1137	5741	548412	
	simple_egg_sam_strict_2n	30	11533	45056	43	84	436	2148	245973	
	simple_egg_sam_strict_3n	30	1122	2840	22	76	144	456	13817	
	simple_tgp_multi_strict	30	40	19	11	24	36	53	91	
	simple_tgp_single_strict	30	66	29	24	43	64	76	149	
strict	9	simple_egg_sam_strict_1n	0	-	-	-	-	-	-	-
		simple_egg_sam_strict_2n	30	9890	29042	34	96	627	6977	158233
		simple_egg_sam_strict_3n	30	768	2076	13	49	104	218	8521
		simple_tgp_multi_strict	30	47	16	17	40	47	56	87
		simple_tgp_single_strict	30	69	30	36	49	57	78	166
	10	simple_egg_sam_strict_1n	0	-	-	-	-	-	-	-
		simple_egg_sam_strict_2n	30	7539	20298	19	120	517	1982	88716
		simple_egg_sam_strict_3n	30	2561	5308	40	112	306	1645	22711
		simple_tgp_multi_strict	30	59	25	27	38	56	68	121
		simple_tgp_single_strict	30	83	37	21	57	71	105	165
non-strict	11	simple_egg_sam_nonstrict_1n	0	-	-	-	-	-	-	-
		simple_egg_sam_nonstrict_2n	30	21380	45778	121	544	6241	15808	232758
		simple_egg_sam_nonstrict_3n	30	15031	50296	26	128	272	3407	271033
		simple_tgp_multi_strict	30	78	39	24	54	70	91	210
		simple_tgp_single_strict	30	87	27	31	72	84	103	144
	12	simple_egg_sam_nonstrict_1n	0	-	-	-	-	-	-	-
		simple_egg_sam_nonstrict_2n	28	29393	38093	72	707	13242	40215	136393
		simple_egg_sam_nonstrict_3n	30	6828	16539	50	292	481	3152	77503
		simple_tgp_multi_strict	30	81	25	36	62	80	98	129
		simple_tgp_single_strict	30	122	52	61	80	114	144	263
non-strict	13	simple_egg_sam_nonstrict_1n	0	-	-	-	-	-	-	-
		simple_egg_sam_nonstrict_2n	30	48360	52182	145	10518	36008	57160	198080
		simple_egg_sam_nonstrict_3n	30	14865	38763	75	350	1405	5457	175512
		simple_tgp_multi_strict	30	81	32	22	59	79	97	161
		simple_tgp_single_strict	30	111	50	43	78	101	136	222
	14	simple_egg_sam_nonstrict_1n	0	-	-	-	-	-	-	-
		simple_egg_sam_nonstrict_2n	22	34233	34499	273	3663	23192	52189	105703
		simple_egg_sam_nonstrict_3n	29	12313	18914	170	649	1603	18752	64404
		simple_tgp_multi_strict	30	100	34	49	78	94	116	189
		simple_tgp_single_strict	30	137	37	88	112	135	158	225
non-strict	15	simple_egg_sam_nonstrict_1n	0	-	-	-	-	-	-	-
		simple_egg_sam_nonstrict_2n	18	9820	17151	424	1338	2954	5126	55660
		simple_egg_sam_nonstrict_3n	27	8812	14289	131	406	1103	10978	43166
		simple_tgp_multi_strict	30	113	43	51	87	108	128	214
		simple_tgp_single_strict	30	137	56	50	85	133	168	289
	3	simple_egg_sam_nonstrict_1n	30	11	15	0	1	7	14	52
		simple_egg_sam_nonstrict_2n	30	3	5	0	0	2	5	23
		simple_egg_sam_nonstrict_3n	30	4	6	0	0	1	4	29
		simple_tgp_multi_nonstrict	30	10	7	1	5	8	14	31
		simple_tgp_single_nonstrict	30	14	9	1	6	12	22	42
4	simple_egg_sam_nonstrict_1n	30	43	41	2	18	29	52	200	
	simple_egg_sam_nonstrict_2n	30	17	21	0	4	9	20	75	
	simple_egg_sam_nonstrict_3n	30	15	23	0	2	6	17	101	
	simple_tgp_multi_nonstrict	30	14	8	2	7	12	18	32	
	simple_tgp_single_nonstrict	30	22	16	4	11	16	28	71	
5	simple_egg_sam_nonstrict_1n	30	128	169	2	33	68	109	710	
	simple_egg_sam_nonstrict_2n	30	62	103	0	19	25	53	493	
	simple_egg_sam_nonstrict_3n	30	26	29	0	10	17	34	148	
	simple_tgp_multi_nonstrict	30	21	10	8	13	18	29	42	
	simple_tgp_single_nonstrict	30	28	14	7	19	24	37	58	
6	simple_egg_sam_nonstrict_1n	30	213	179	25	72	131	325	717	
	simple_egg_sam_nonstrict_2n	30	66	45	0	39	58	86	181	
	simple_egg_sam_nonstrict_3n	30	45	36	10	20	35	61	157	
	simple_tgp_multi_nonstrict	30	27	15	5	15	24	40	60	
	simple_tgp_single_nonstrict	30	34	16	11	22	32	46	84	
7	simple_egg_sam_nonstrict_1n	30	319	303	26	134	188	438	1436	
	simple_egg_sam_nonstrict_2n	30	115	98	6	45	98	138	386	
	simple_egg_sam_nonstrict_3n	30	64	61	1	28	46	71	281	
	simple_tgp_multi_nonstrict	30	35	16	6	27	32	48	74	
	simple_tgp_single_nonstrict	30	54	32	26	33	44	66	187	
8	simple_egg_sam_nonstrict_1n	30	623	732	56	254	396	696	3812	
	simple_egg_sam_nonstrict_2n	30	150	127	5	57	104	226	525	
	simple_egg_sam_nonstrict_3n	30	118	135	18	34	70	130	608	
	simple_tgp_multi_nonstrict	30	38	16	17	25	38	50	67	
	simple_tgp_single_nonstrict	30	67	33	17	46	60	84	164	
9	simple_egg_sam_nonstrict_1n	30	1248	1046	95	294	1124	2025	3372	
	simple_egg_sam_nonstrict_2n	30	296	225	43	118	196	444	967	
	simple_egg_sam_nonstrict_3n	30	139	110	22	64	104	192	524	
	simple_tgp_multi_nonstrict	30	49	27	15	33	44	58	166	
	simple_tgp_single_nonstrict	30	79	36	32	52	70	94	191	
10	simple_egg_sam_nonstrict_1n	30	1419	1042	152	701	1311	1679	3886	
	simple_egg_sam_nonstrict_2n	30	326	234	41	153	264	404	1015	
	simple_egg_sam_nonstrict_3n	30	295	399	49	110	161	256	1792	
	simple_tgp_multi_nonstrict	30	52	19	16	38	48	63	101	
	simple_tgp_single_nonstrict	30	86	33	32	64	80	99	172	
11	simple_egg_sam_nonstrict_1n	30	2348	1880	329	1081	1970	2909	9072	
	simple_egg_sam_nonstrict_2n	30	375	281	68	191	298	502	1101	
	simple_egg_sam_nonstrict_3n	30	234	199	26	86	166	302	781	
	simple_tgp_multi_nonstrict	30	57	22	20	40	55	70	128	
	simple_tgp_single_nonstrict	30	108	46	47	72	94	134	218	
12	simple_egg_sam_nonstrict_1n	30	2824	3513	240	1185	1772	2374	18617	
	simple_egg_sam_nonstrict_2n	30	595	358	60	344	528	790	1545	
	simple_egg_sam_nonstrict_3n	30	317	233	35	127	242	465	806	
	simple_tgp_multi_nonstrict	30	69	26	28	52	67	81	126	
	simple_tgp_single_nonstrict	30	112	41	51	80	100	141	206	
13	simple_egg_sam_nonstrict_1n	30	2732	1854	440	1395	2344	3506	8635	
	simple_egg_sam_nonstrict_2n	30	969	1097	44	272	714	1345	5627	
	simple_egg_sam_nonstrict_3n	30	424	378	38	158	298	472	1492	
	simple_tgp_multi_nonstrict	30	71	28	31	53	63	87	127	
	simple_tgp_single_nonstrict	30	120	40	45	91	108	141	241	
14	simple_egg_sam_nonstrict_1n	30	4608	3438	767	2576	3742	6440	15033	
	simple_egg_sam_nonstrict_2n	30	807	738	140	396	612	1092	3992	
	simple_egg_sam_nonstrict_3n	30	504	662	55	196	338	457	3030	
	simple_tgp_multi_nonstrict	30	86	30	39	65	80	101	167	
	simple_tgp_single_nonstrict	30	122	39	56	95	122	148	226	
15	simple_egg_sam_nonstrict_1n	30	4782	3219	698	2870	3598	5304	13031	
	simple_egg_sam_nonstrict_2n	30	1093	999	229	532	754	1206	5107	
	simple_egg_sam_nonstrict_3n	30	640	504	111	277	531	815	2240	
	simple_tgp_multi_nonstrict	30	96	21	39	84	96	112	144	
	simple_tgp_single_nonstrict	30	135	43	68	110	121	159	219	